МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине «Основы машинного обучения»

Тема: «Сравнение классических методов классификации»

Выполнил:
студент 2 курса
группы АС-67
Лискович И.Г.

Проверил:
Крощенко А. А.

2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Общий план для всех вариантов:

1. Импорт библиотек и подготовка данных
    - импортируйте torch, torch.nn, torch.optim, а также sklearn для загрузки данных и их предобработки;
    - загрузите датасет, выполните стандартизацию (StandardScaler) и кодирование признаков;
    - разделите данные на обучающую и тестовую выборки;
    - преобразуйте данные (признаки и метки) в тензоры PyTorch: torch.tensor(X_train, dtype=torch.float32).
2. Определение архитектуры нейронной сети
    - создайте класс, наследуемый от torch.nn.Module;
    - в методе __init__ определите все слои, которые будете использовать (например, nn.Linear, nn.ReLU, nn.Dropout);
    - в методе forward опишите последовательность применения слоев к входным данным.
3. Инициализация модели, функции потерь и оптимизатора
    - создайте экземпляр вашей модели: model = MLP();
    - определите функцию потерь. Для бинарной классификации используйте nn.BCEWithLogitsLoss, для многоклассовой – nn.CrossEntropyLoss;
    - определите оптимизатор: optimizer = torch.optim.Adam(model.parameters(), lr=0.001).
4. Написание цикла обучения (Training Loop)
    - запустите цикл на определенное количество эпох;
    - внутри цикла:
        1. переведите модель в режим обучения: model.train();
        2. сделайте предсказание (forward pass): y_pred = model(X_train);
        3. рассчитайте потери (loss): loss = criterion(y_pred, y_train);
        4. обнулите градиенты: optimizer.zero_grad();
        5. выполните обратное распространение ошибки: loss.backward();
        6. сделайте шаг оптимизации: optimizer.step().
5. Оценка модели (Evaluation)
    - переведите модель в режим оценки: model.eval();

    - используйте with torch.no_grad():, чтобы отключить расчет градиентов;
    - сделайте предсказания на тестовых данных;
    - преобразуйте выходные данные (логиты) в предсказанные классы (например, с помощью torch.argmax или проверки порога > 0);
    - рассчитайте метрики (accuracy, f1-score и т.д.), используя sklearn.metrics.

Код:

```
import torch
from torch import nn
from torch import optim

import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

DATASET_PATH = "telco_customer_churn.csv"

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Платформа обучения:", device)

torch.manual_seed(42)
np.random.seed(42)


class MLP(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int = 24, dropout:
float = 0.0) -> None:
        super().__init__()

        layers = [
            nn.Linear(in_features=input_dim, out_features=hidden_dim),
            nn.ReLU(),
        ]

        if dropout > 0:
            layers.append(nn.Dropout(p=dropout))

        layers.append(nn.Linear(in_features=hidden_dim, out_features=1))

        self.linear_relu_stack = nn.Sequential(*layers)
```

```
    def forward(self, x):
        return self.linear_relu_stack(x)


def load_and_prepare_data():
    df = pd.read_csv(DATASET_PATH, sep=",")

    if "customerID" in df.columns:
        df = df.drop(columns=["customerID"])

    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"],
errors="coerce")

    y = (df["Churn"] == "Yes").astype(np.int32).values
    X = df.drop(columns=["Churn"])

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.25,
        random_state=42,
        stratify=y,
    )

    numeric_cols =
X_train.select_dtypes(exclude=["object"]).columns.tolist()
    categorical_cols =
X_train.select_dtypes(include=["object"]).columns.tolist()

    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]
    )

    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
```

```
            ("onehot", OneHotEncoder(handle_unknown="ignore",
sparse_output=False)),
        ]
    )

    preprocessor = ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, numeric_cols),
            ("cat", categorical_transformer, categorical_cols),
        ],
        sparse_threshold=0,
    )

    X_train_scaled = preprocessor.fit_transform(X_train)
    X_test_scaled = preprocessor.transform(X_test)

    X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32)
    X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32)

    y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1,
1)

    return X_train_tensor, X_test_tensor, y_train_tensor, y_test


def train_and_evaluate(
    X_train_tensor,
    X_test_tensor,
    y_train_tensor,
    y_test,
    dropout: float,
    experiment_name: str,
    epochs: int = 50,
):
    model = MLP(
        input_dim=X_train_tensor.shape[1],
        hidden_dim=24,
        dropout=dropout,
    ).to(device)
```

```
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    print(f"Эксперимент: {experiment_name}")
    print("Архитектура модели:")
    print(model)
    print(f"Количество эпох: {epochs}")
    print("Оптимизатор: Adam, lr=0.001")
    print("Функция потерь: BCE WithLogitsLoss")

    for epoch in range(epochs):
        model.train()

        y_pred = model(X_train_tensor)
        loss = criterion(y_pred, y_train_tensor)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if (epoch + 1) % 10 == 0:
            print(f"Эпоха {epoch + 1:03d}/{epochs}, loss =
{loss.item():.4f}")

    model.eval()
    with torch.no_grad():
        logits = model(X_test_tensor)
        probabilities = torch.sigmoid(logits)
        predicted_classes = (probabilities >=
0.5).int().cpu().numpy().ravel()

    print("\nОценка модели:")
    print(
        classification_report(
            y_test,
            predicted_classes,
            target_names=["No churn", "Churn"],
            digits=4,
        )
    )
```

```
    return model, predicted_classes


def solution() -> None:
    X_train_tensor, X_test_tensor, y_train_tensor, y_test =
load_and_prepare_data()

    print("Размер обучающей выборки:", X_train_tensor.shape)
    print("Размер тестовой выборки:", X_test_tensor.shape)
    print("Количество признаков после кодирования:",
X_train_tensor.shape[1])

    train_and_evaluate(
        X_train_tensor,
        X_test_tensor,
        y_train_tensor,
        y_test,
        dropout=0.0,
        experiment_name="Базовая модель без Dropout",
        epochs=50,
    )

    train_and_evaluate(
        X_train_tensor,
        X_test_tensor,
        y_train_tensor,
        y_test,
        dropout=0.2,
        experiment_name="Модель с Dropout(0.2) после скрытого слоя",
        epochs=50,
    )

    print("Сравните classification_report для двух экспериментов выше.")
    print("Dropout помогает, если метрики на тесте выросли или модель
стала устойчивее.")

if __name__ == "__main__":
    solution()
```

Вывод: Я узнал как деалется нейросеть.